4-bit Array Multiplier: RTL to GDSII with Open-Source Tools

Digital VLSI Design — SkyWater SKY130 (`sky130_fd_sc_hd`), IIC-OSIC-TOOLS (Yosys 0.70, OpenSTA 3.1.0, LibreLane 3.1.0.dev3)

Team: Vedant Tejas (2023112018) and Aarav Singla (2024122004)

Repository: github.com/Starkdoorstep12/mult4-rtl2gds

Part A: RTL design and verification

Files: `rtl/full_adder.v`, `rtl/mult_array.v` (12 `full_adder` instances in three generate loops, no `*` operator, asynchronous active-low reset, 16 flip-flops, latency 2 cycles) and `tb/tb_mult.v` (exhaustive, self-checking, 256 vectors, writes `mult.vcd`).

Simulation result (RTL)

```
PASS: 256 vectors checked, 0 errors
```

Testbench check (task 6): `sum` in `full_adder.v` was temporarily changed to `a ^ b`. The testbench then reports 95 mismatches (for example a=15, b=15 expected 225, got 181) and ends with:

```
FAIL: 256 vectors checked, 95 errors
```

The change was undone afterwards and the design passes again.

Hand-worked example (task 5)

A = last digit of Vedant Tejas's roll number (2023112018, last digit 8) + 5 = 13, and B = 7 (A is not 7). Vedant's roll number was used.

```
Bit:          7 6 5 4 3 2 1 0
                    1 1 0 1     A = 13
                x   0 1 1 1     B = 7
              ------------------
pp0  (b0=1)             1 1 0 1
pp1  (b1=1)           1 1 0 1       shifted 1
pp2  (b2=1)         1 1 0 1         shifted 2
pp3  (b3=0)       0 0 0 0           shifted 3
              ------------------
P             0 1 0 1 1 0 1 1     = 91  (0x5B)
```

Check: $13 + 26 + 52 + 0 = 91 = 0\text{b}01011011 = 0\text{x}5B$.

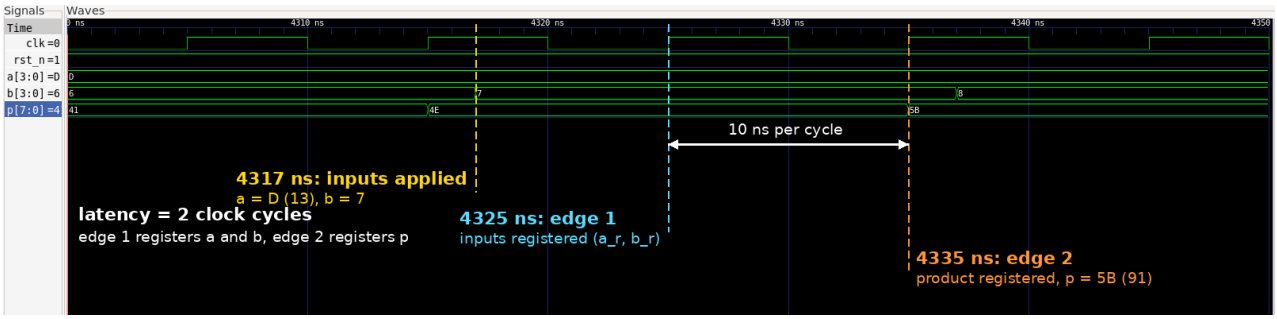


Figure 1: Waveform in GTKWave for A = 13 (D), B = 7 (screenshot, with marker lines and labels added). The inputs are applied at 4317 ns and registered at edge 1 (4325 ns). The product 5B (91) is registered at edge 2 (4335 ns). Latency = 2 clock cycles.

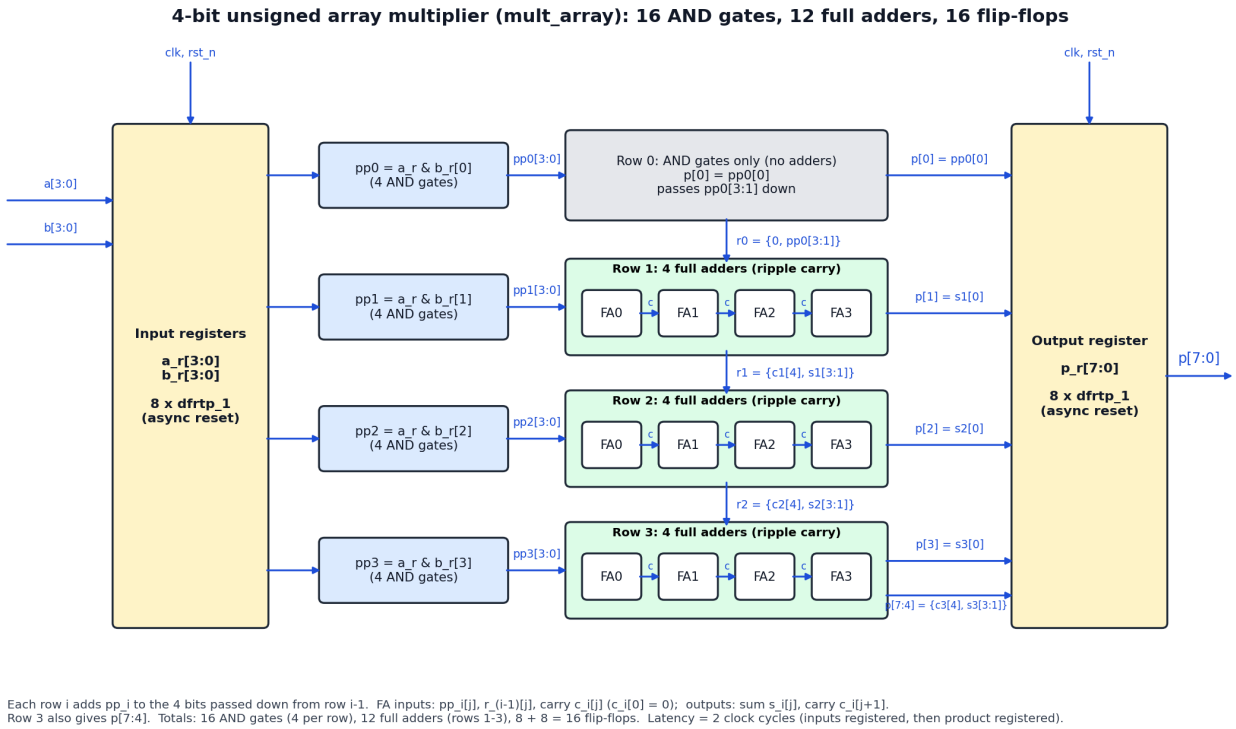


Figure 2: Block diagram of `mult_array`: 16 AND gates (4 per row), 12 full adders (rows 1 to 3), and 8 input plus 8 output flip-flops.

Part B: Logic synthesis with Yosys

Metric	Value
Total number of cells	61
Number of flip-flops	16 (sky130_fd_sc_hd__dfrtp_1)
Total cell area (um2)	756.976
Flip-flop area (um2)	400.384 (16 x 25.024)
Flip-flop area (% of total)	52.89
3 most-used cell types	dfrtp_1 x16, nand2_1 x7, then a tie: and2_0 x4 and maj3_1 x4 (logic only: nand2_1 x7, and2_0 x4, maj3_1 x4)
Netlist simulation result	PASS: 256 vectors checked, 0 errors

Command: `yosys -c synth/synth.tcl`. Gate-level simulation of `synth/mult_array_netlist.v` with the assignment's `iverilog` command (`-DFUNCTIONAL -DUNIT_DELAY=#1`) prints `PASS: 256 vectors checked, 0 errors`.

Part C: Static timing analysis with OpenSTA

Metric	Value
Worst slack at T = 10 ns (ns)	7.21
Tmin (ns)	2.79 (= 10 - 7.21)
Fmax (MHz)	358.4 (= 1000 / 2.79)
Slack at T = Tmin (ns)	+0.003 (MET)
Critical path start point	_91_ (dfrtp_1), input register a_r[1], clocked by clk
Critical path end point	_88_ (dfrtp_1) D pin, output register p[6]
Number of cells on the critical path	9 (launch flip-flop + 8 logic cells), 10 counting the capture flip-flop
Latency (ns) = 2 x Tmin	5.58
Throughput (million results per second)	358.4

Critical path cells, in order (typical corner, T = 2.79 ns)

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role
1	_91_	dfrtp_1 (CLK to Q)	0.398	0.398	launch register, a_r[1]
2	_37_	clkinv_1	0.073	0.471	inverts a_r[1]
3	_42_	o311ai_0	0.356	0.827	partial products of a_r[1:0], b_r[1:0] merged with row-1 logic
4	_45_	a21oi_1	0.153	0.980	row-1 carry logic
5	_51_	maj3_1	0.402	1.382	full-adder carry
6	_58_	xnor3_1	0.370	1.752	full-adder sum
7	_61_	maj3_1	0.368	2.119	full-adder carry
8	_63_	maj3_1	0.354	2.474	full-adder carry
9	_64_	xor3_1	0.178	2.652	full-adder sum, p_next[6]
-	_88_	dfrtp_1 (D, setup 0.135)	-	required 2.655	capture register, p[6]

Cells that implement a full-adder function: 5 (3 maj3 carries, 1 xnor3 sum, 1 xor3 sum).

Logic delay = 2.652 - 0.398 = 2.254 ns. Clock-to-Q (0.398) + setup (0.135) = 0.533 ns of overhead.

Note: the report also lists the asynchronous reset recovery check (rst_n to _82_/RESET_B), slack 2.018 ns. It is not the critical path.

Part D: Physical design (RTL to GDSII with LibreLane)

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10 (3.5 ns up to 8.0 ns in 0.5 ns steps), plus one 10 ns trial run
Core area (um2)	17759.5 (die 22500)
Cells after synthesis	61
Cells after place and route	1883
Kinds of cells added	decap (1298), fill (254), tap (245), timing-repair buffers (21), clock buffers (3)
Wirelength	2499 um (detailed routing log; met1 1153, met2 1284, met3 61)
Setup slack (ns)	+0.396 (worst corner, ss_100C_1v60); +4.207 at the typical corner (max_tt_025C_1v80)
Post-route Fmax (MHz)	131.5 from the worst corner; 263.6 from the typical corner
Fmax from Part C (MHz)	358.4 (Tmin 2.79 ns, typical corner)
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

Cell count check: 1883 = 62 original-design cells (16 flip-flops + 46 logic cells; synthesis gave 16 + 45) + 1552 decap/fill + 245 tap + 21 repair buffers + 3 clock buffers.

All attempts

`config.json` sets `DIE_AREA` to [0, 0, 150, 150] and `FP_SIZING` to `absolute`. Attempts 1 to 10 start at 1.2 × Tmin = 3.35 ns, rounded up to 3.5 ns, and add 0.5 ns after each failure. Slack is the worst across all corners.

Attempt	CLOCK_PERIOD (ns)	DRC	LVS	Setup slack (ns)	Result
trial	10	0	0	+2.396	Pass (flow check only)
1	3.5	0	0	−2.188	Fail (setup)
2	4.0	0	0	−1.688	Fail
3	4.5	0	0	−1.188	Fail
4	5.0	0	0	−0.686	Fail
5	5.5	0	0	−0.208	Fail
6	6.0	0	0	−0.154	Fail
7	6.5	0	0	−0.220	Fail
8	7.0	0	0	−0.134	Fail
9	7.5	0	0	−0.104	Fail
10	8.0	0	0	+0.396	Pass: final run

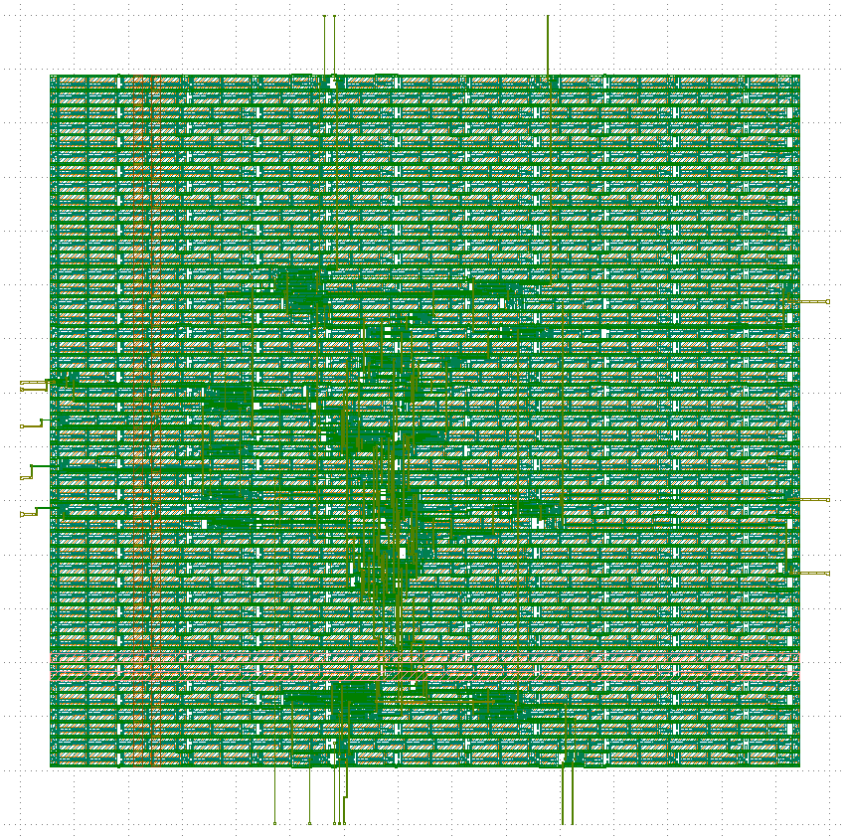


Figure 3: Final layout of `pnr/mult_array.gds` (8.0 ns run) in KLayout: the whole die (150 × 150 µm; the cell area is about 139 µm wide). Layers shown: diffusion (65/20), poly (66/20), li1 (67/20) and met1 to met5 (68/20 to 72/20); text labels are hidden. Standard cell rows, the power stripes, the signal routing and the I/O pins are visible.

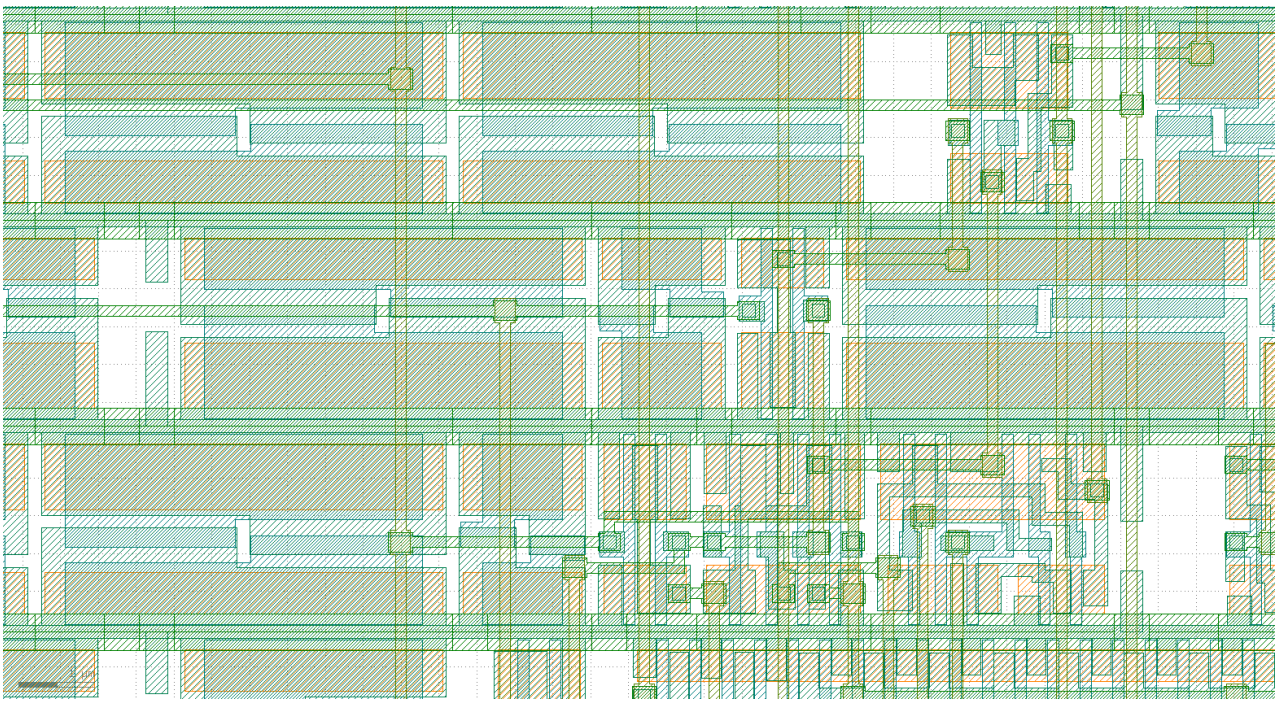


Figure 4: Zoomed view in KLayout, roughly 30 × 18 µm in the densest routing area (same layers). The horizontal met1 power rails, one row pitch (2.72 µm) apart, mark the standard cell rows. li1, poly and diffusion are visible inside the cells, with met1 and met2 wires and the vias between them.

Answers to the questions

Q1. Why is the logic not simply 16 + 12 cells?

Synthesis produced 61 cells in total. Subtracting the 16 flip-flops leaves 45 logic cells, not 16 + 12 = 28. The assignment's count is the number of building blocks (AND gates and full adders), not library cells. Yosys flattened the full adders, and ABC then remapped the whole logic cloud as one network. The stat report shows only 4 maj3 cells (a carry-like function) and a mix of xor3, xnor3, xor2, xnor2, nand, and and compound cells such as a21oi and a22oi, rather than 12 identical adder blocks. Also, the first adder of each row has a constant-zero carry-in, and the top bit of the row-0 output is constant zero, so synthesis can simplify those adders into simpler gates. ABC also merges AND gates into compound cells.

Q2. Critical path

OpenSTA (typical corner, tt_025C_1v80) gives a worst slack of 7.21 ns at T = 10 ns, so Tmin = 10 - 7.21 = 2.79 ns and Fmax = 1000 / 2.79 = 358.4 MHz; the rerun at T = 2.79 ns has +0.003 ns slack. The critical path starts at the input register for a[1] (`_91_`, a_r[1]) and ends at the output register for p[6] (`_88_`), so it is register to register, as expected with registered inputs and outputs. It has 9 cells: the launching dfrtp_1 and 8 logic cells (clkinv, o311ai, a21oi, maj3, xnor3, maj3, maj3, xor3), plus the capturing flip-flop. Only 5 of the 8 logic cells are full-adder functions (3 maj3 carries, 1 xnor3 sum, 1 xor3 sum), against the estimate of about 2N = 8 full-adder delays. The path is shorter than the estimate because ABC maps a whole full-adder carry into one maj3 cell and a whole sum into one xor3/xnor3 cell, and because the constant-zero carry-ins of each row and the zero top bit of row 0 turn the first adders into simpler gates, which ABC merged with the AND gates into compound cells (o311ai, a21oi). Of the 2.79 ns, 2.254 ns is logic delay and 0.533 ns is flip-flop clock-to-Q (0.398) plus setup (0.135).

Q3. Flip-flop share of the area

Flip-flops take 400.384 um2 out of 756.976 um2, which is 52.89%. Each dfrtp_1 cell is 25.024 um2, while the average logic cell is only about 7.9 um2 ((756.976 - 400.384) / 45), so one flip-flop is roughly three times the size of an average logic cell. The design is tiny, with only 45 small logic cells, but it registers all 8 input bits and all 8 output bits, so the 16 flip-flops are a large share of the total.

Q4. Why is post-route Fmax lower, and which cells were added?

Part C is an idealised estimate: typical corner only, an ideal clock, no wire parasitics and a 1.0 ns input and

output delay. It gives Tmin = 2.79 ns, i.e. 358.4 MHz. After place and route, LibreLane checks several corners with its own constraints, real wire parasitics, a built clock tree and resized cells. In the typical corner the worst slack at the 8.0 ns period is +4.207 ns, which is a worst-path delay of 3.793 ns (263.6 MHz) under LibreLane's constraints. The signed-off figure comes from the slow corner (ss_100C_1v60), where the worst slack is +0.396 ns, so post-route Fmax = 1000 / (8.0 - 0.396) = 131.5 MHz. The slow corner is the limiting one: the 10 ns trial had +2.396 ns there, and the 7.5 ns run failed with -0.104 ns. We did not separate how much of the gap between 2.79 ns and 3.793 ns comes from wires, the clock tree, cell resizing or the different constraints. The flow took the design from 61 to 1883 cells by adding decap cells (1298), fill cells (254), tap cells (245), timing-repair buffers (21) and clock buffers (3). Tap cells tie the wells to the supply rails to prevent latch-up. Decap cells give the supply local charge to limit voltage droop. Fill cells keep the cell rows continuous so the well and implant layers are complete. Repair buffers fix slew and timing violations. Clock buffers balance the clock arrival at the 16 flip-flops.

Q5. DRC and LVS

DRC (design rule check) checks the physical layout geometry against the foundry's manufacturing rules, such as minimum wire width, spacing between shapes, and via enclosure, so the layout can actually be made. LVS (layout versus schematic) extracts the devices and connections from the layout and compares them with the intended netlist, so the layout implements the circuit we designed. Both must pass: a layout can obey every rule and still be the wrong circuit (DRC clean, LVS fail), or be the right circuit but unmanufacturable (LVS clean, DRC fail). In our final 8.0 ns run both reported 0 errors (Magic DRC 0, KLayout DRC 0, LVS 0), and every earlier attempt also reported 0 routing DRC errors and 0 LVS errors.

Q6. Scaling to 8 bits

From Section 2, an N-bit array multiplier needs N^2 AND gates and N(N-1) full adders, and its longest path is about 2N full-adder delays. For N = 4: 16 AND gates, 4 x 3 = 12 full adders, about 8 full-adder delays. For N = 8: 64 AND gates, 8 x 7 = 56 full adders, about 16 full-adder delays. So the adder count grows by 56 / 12 = 4.7 times (it grows as N^2), while the critical path only doubles (it grows linearly with N). Using our own numbers: the logic part of our critical path is 2.254 ns, so doubling it gives about 4.51 ns; adding the same flip-flop overhead of 0.533 ns gives Tmin of about 5.04 ns, so Fmax would fall from 358.4 MHz to about 1000 / 5.04 = 198 MHz. This is a first-order estimate; the real value depends on how synthesis maps the larger array.

Individual contributions

The work was divided in a team contract before it started, and each of us reviewed the other's parts. The table lists who did what; the Git history (`git log`) records the commits of each author.

Area	Vedant Tejas (2023112018)	Aarav Singla (2024122004)
Repository, README	Created the repository and folder structure. README: structure, commands for Parts B and D and the screenshots, and all ten LibreLane attempts. Team contract.	README commands logs for Parts A and C.
RTL	Wrote `rtl/full_adder.v` and `rtl/mult_array.v`.	Checked `mult_array.v` against the connection table in Section 2 of the assignment.
Testbench and FAIL test	Reviewed the testbench; ran it against the RTL and the synthesized netlist (both PASS).	Wrote `tb/tb_mult.v`. Ran the FAIL test with `sum = a ^ b` (95 errors).
Part B: synthesis	Wrote `synth/synth.tcl`; ran Yosys; netlist, stat report and table; gate-level simulation.	Used the synthesized netlist as the input to Part C.
Part C: static timing	Cross-check: re-ran Aarav's script at T = 10 ns (worst slack 7.21 ns) and obtained the same Tmin from a separate scratch script.	Wrote `sta/sta.tcl`; found Tmin = 2.79 ns (Fmax 358.4 MHz) and the critical path; Part C table; Q2.
Part D: physical design	Wrote `pnr/config.json`; ran LibreLane (ten attempts from 3.5 to 8.0 ns); `metrics.json`, GDS and the Part D table.	Reviewed the Part D results; raised the Q4 caveat about LibreLane's own I/O constraints.
Screenshots	Took the GTKWave waveform and the two KLayout screenshots (the GUI tools did not open on Aarav's Mac).	Not applicable (see left).
Hand example, block diagram	Block diagram (generated with a Python script); hand-worked example (Vedant's roll number).	Worked out the same hand example independently (13 x 7 = 91).
Report	Q1, Q3, Q4, Q5; Part B and D tables; built and compiled the LaTeX report.	Q2, Q6; Part C table; reviewed the repository and report and reported fixes (empty `config.json`, missing roll numbers, Q5 wording, README commands).